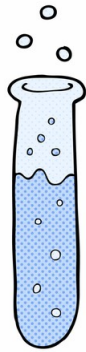


Lecture 3

Testing & Greedy Algorithms



Announcements

- ❖ No Lab on next Monday
 - Labor Day!
- ❖ Office hours are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group

Big Picture

- ❖ **This week:** getting up to speed with Python

- OOP & Testing

- ❖ **Next week:** start thinking algorithmically

- Greedy algorithms & empirical benchmarking
- F: Project 1 topics released

- ❖ **Next next week:** running time analysis

- Growth rates & Big-O
- F: Project 1 Proposal Due

Today

- ❖ Testing in Python
 - Unit tests
 - Test-Driven Development
- ❖ Introduce Greedy Algorithms
 - First algorithmic paradigm

Testing

- ❖ Essential practice for producing reliable, working software
- ❖ Types of tests:
 - Unit tests: individual object, function, or behavior
 - Integration tests: different components of a system interact
 - System tests: end-to-end behavior of software
 - ...

Unit Testing

- ❖ Suppose our codebase has function `sumOdd` that sums the odd numbers from a list
- ❖ What are some cases we may want to test?

Unit Testing

- ❖ Suppose our codebase has function `sumOdd` that sums the odd numbers from a list
- ❖ What are some cases we may want to test?
 - List of odd and even numbers (positive example)
 - List of odd numbers (positive example)
 - List of even numbers (positive example)
 - List of strings (negative example)
 - Empty list (edge case)
 - Some other data type (negative example)

Unit Testing

- ❖ Suppose our codebase has function `sumOdd` that sums the odd numbers from a list
- ❖ What are some cases we may want to test?
 - List of odd and even numbers (positive example)
 - List of odd numbers (positive example)
 - List of even numbers (positive example)
 - List of strings (negative example)
 - Empty list (edge case)
 - Some other data type (negative example)

Frameworks

- `unittest`
- `pytest`

Test-Driven Development (TDD)

- ❖ Engineering strategy of planning code through tests
- ❖ **First:** map out your functions/classes through tests
- ❖ **Next:** incrementally implement them until all the tests pass

Test-Driven Development (TDD)

- ❖ Engineering strategy of planning code through tests
- ❖ **First:** map out your functions/classes through tests
- ❖ **Next:** incrementally implement them until all the tests pass
- ❖ Most often used for large, mission-critical projects (where failure is costly)
- ❖ Hybrid approach between pre/post testing is often the best practice
 - (my opinion)

Why test?

- ❖ What sorts of issues are we looking for?

Why test?

- ❖ What sorts of issues are we looking for?
 - Logic-based issues
 - Code-based issues

Why test?

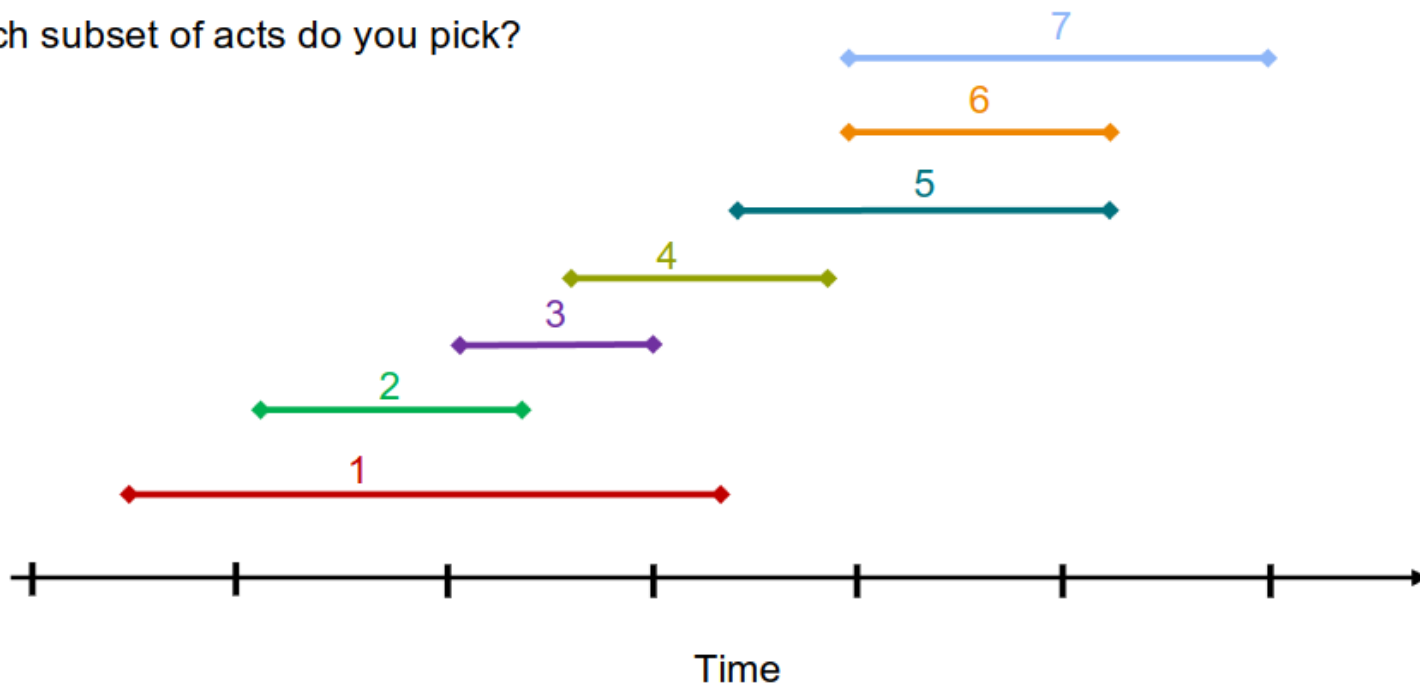
- ❖ What sorts of issues are we looking for?
 - Logic-based issues
 - Code-based issues
- ❖ Brings us back to the distinction between an algorithm and its implementation
- ❖ We're focusing mostly on the algorithm aspect

Greedy Algorithms

Interval Scheduling Problem

- ❖ Imagine you are going to a music festival with the goal of seeing as many acts as possible

Which subset of acts do you pick?

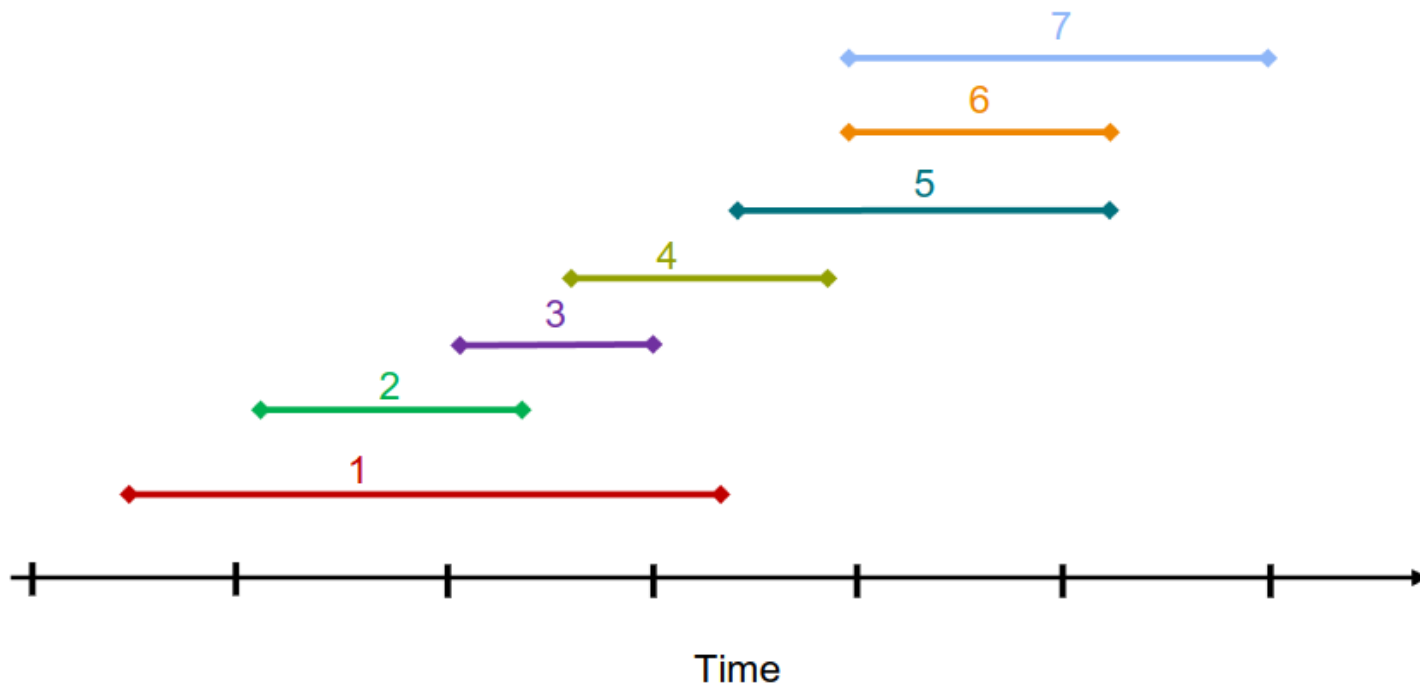


Interval Scheduling Problem

- ❖ Let's introduce some notation:
 - Acts $1, 2, \dots, n$
 - $s(i)$ for the start time of act i
 - $f(i)$ for the finish time of act i
 - Acts i, j are **compatible** if they do not overlap
 - A collection of acts A are **compatible** if all pairs in A are compatible
 - A collection of acts A is **optimal** if A is compatible and no other collection is larger than A

Interval Scheduling Problem

- ❖ **Goal:** find an optimal collection of acts



Baseline Solution

- ❖ What's a simple solution that is guaranteed to solve this problem but may be inefficient?

Baseline Solution

- ❖ What's a simple solution that is guaranteed to solve this problem but may be inefficient?
 - Enumerate every set
 - For each: check if it is consistent and, if so, note its size
- ❖ How many checks does this require?
 - Recall: there are n acts

Baseline Solution

- ❖ What's a simple solution that is guaranteed to solve this problem but may be inefficient?
 - Enumerate every set
 - For each: check if it is consistent and, if so, note its size
- ❖ How many checks does this require?
 - Recall: there are n acts

2^n checks

Greedy Algorithms

- ❖ Make choices one at a time
 - Roughly: choose the thing that looks best and never look back
 - We will sort acts into some "natural order" and choose acts one-by-one if they are compatible with the acts already chosen

Greedy Algorithms

- ❖ Make choices one at a time
 - Roughly: choose the thing that looks best and never look back
 - We will sort acts into some "natural order" and choose acts one-by-one if they are compatible with the acts already chosen

Algorithm sketch:

$R \leftarrow$ set of all shows sorted by some property

$A \leftarrow \emptyset$

while R is not empty **do**

 take first show i from R

 add i to A

 delete i and all overlapping acts from R

Closing Thoughts

- ❖ Is there a natural order that solves the interval scheduling problem?
- ❖ Supposing there is one, is it faster than the baseline solution?

Next Time

- ❖ More on Greedy algorithms